

Object, Class & Encapsulation

Pertemuan 2

Zulkifli Arsyad, S.Kom., M.T.

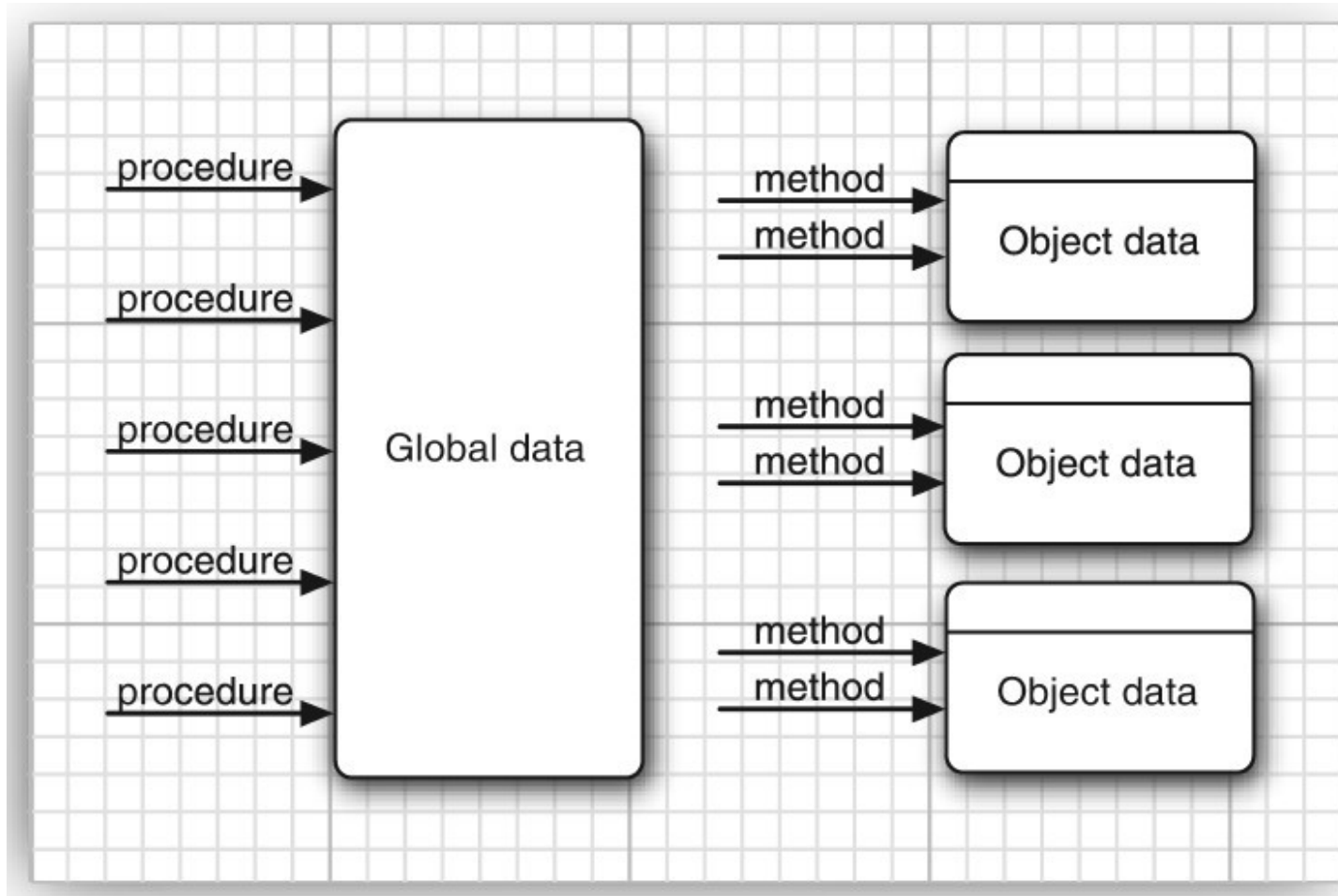


Topics

1. Object
2. Messages
3. Class (class as a complete ADT)
4. Encapsulation



Procedural vs OO Programming



1. Object

- Objek merupakan sesuatu (**tangible** / **intangible**) yang memiliki **state** (attributes) dan **behaviour** (methods).
- Objek disebut juga “**instance of class**” atau representasi dari sebuah kelas.



Objek (contd.)

- An object represent an entity in the real world that can be distinctly identified. For example : a student, a desk, a circle, a button, etc.
- Interaksi antar objek dilakukan melalui “**sending message**”.



3 key characters of objects :

The object's behavior – what can you do with this object, or what method can you apply ?

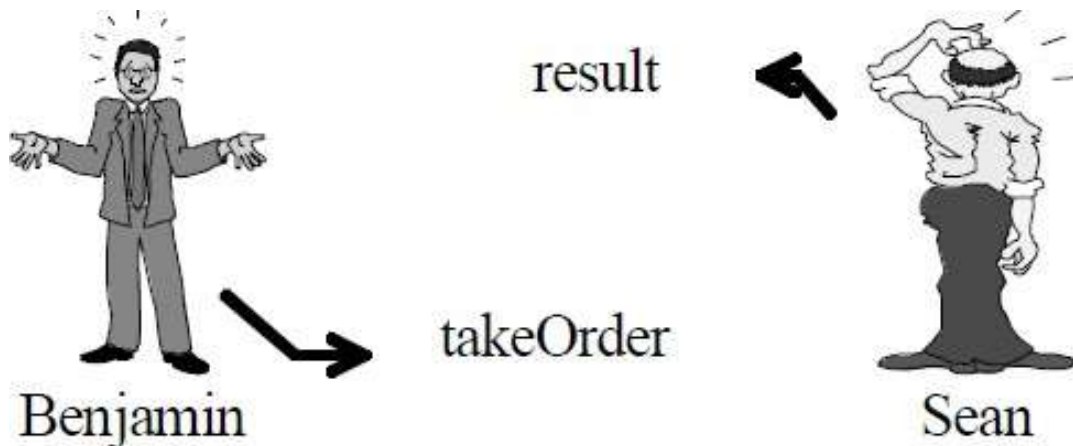
The object's state – how does the object react when you invoke those methods ?

The object's identity – how is the object distinguished from others ?



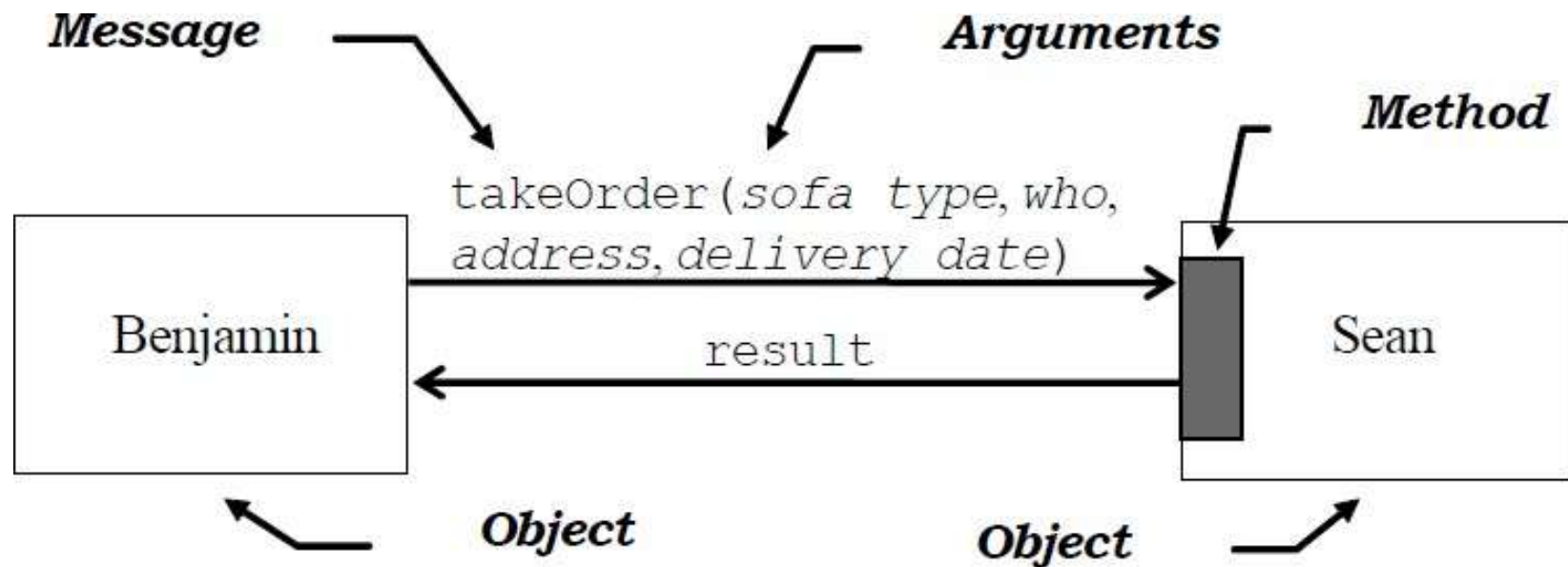
Contoh interaksi antar objek

- Benjamin ingin memesan kursi sofa warna hijau, 5 seater-set, dan dikirimkan pada hari rabu.
- Sean merupakan sales dari toko furniture.
- Bean mengirimkan pesan “takeOrder”, lalu Sean akan mengembalikan “result” sesuai pesanan.



Contd..

- Pesan : **takeOrder**(sofa type, who, address, delivery date)



Simulation

- Berdasarkan kasus tersebut, dapat dikatakan bahwa objek, message dan method menyediakan representasi yang mirip dengan objek pada dunia nyata dan interaksinya.
- OOP memodelkan permasalahan dunia nyata sehingga dapat disimulasikan.
- Desain OOP pertama didesain awal tahun 1970 dengan Bahasa SIMULA



2. Message

- Message merupakan suatu pemanggilan method (**method call**) dari pengirim pesan (**sender**) kepada penerima pesan (**receiver**).
- Informasi tambahan yang dikirimkan melalui “method call” disebut **arguments**.



Message Component

1. **Object identifier** : objek yang menerima pesan (receiver)
2. **Method name** : nama method yang dipanggil
3. **Arguments** : informasi tambahan yang dikirimkan melalui method, biasanya berupa input parameter.



Contd..

Benjamin as an Object

Attributes:

```
name = "Benjamin"  
address = "1, Robinson Road"  
budget = "2000"
```

Methods:

```
purchase() {  
    Sean.takeOrder("Benjamin", "sofa", "1, Robinson Road",  
                  "12 November")  
}  
getBudget() {return budget}
```

The message `Sean.takeOrder(who, stock, address, date)` is interpreted as follows:

- Sean is the receiver of the message;
- `takeOrder` is a method call on Sean;
- `"Benjamin", "stock", "address", "date"` are arguments of the message.

Latihan (10 menit)

- Consider the following scenarios and outline the objects and their interactions in term of messages and arguments :
 - A customer making a cash withdrawal from ATM
 - A customer buying products from a vending machine
 - Buying flower from a florist
 - Etc..



3. Class

- Class merupakan suatu **blueprint** dari objek.
- Class is a “**definition of template for structuring and creating objects with the same attributes and same methods**”.
- Class umumnya berupa “nouns” pada suatu domain system.



Contd..

- Sebagai contoh pada system order-processing, beberapa nounsnya adalah sbb :
 - Item
 - Order
 - Shipping address
 - Payment
 - Account
- Hal-hal tersebut dapat diidentifikasi sebagai class.

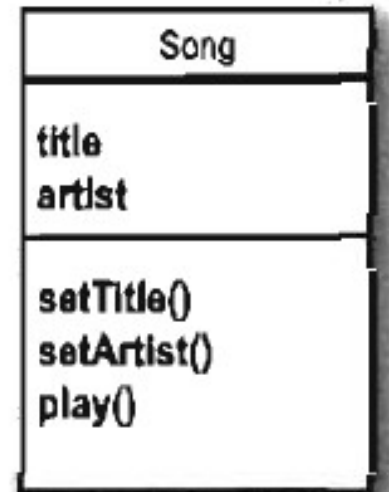


Contd..

- Langkah awal ketika mendesain OOP adalah identifikasikan class dan tentukan state dan behaviornya.
- Struktur kelas terdiri dua, yakni
 1. State (instance variables)
 2. Behaviour (methods)

**instance
variables**
(state)

methods
(behavior)



State

- State dalam lingkup class direpresentasikan dalam bentuk data fields. Sebagai contoh :
 - Class **circle**, memiliki data field : radius
 - Class **rectangle**, memiliki data field : width, dan height
 - Class **car**, memiliki data field : brand, color, police number, type,
- State dapat juga disebut **instance variable** atau **attributes**.

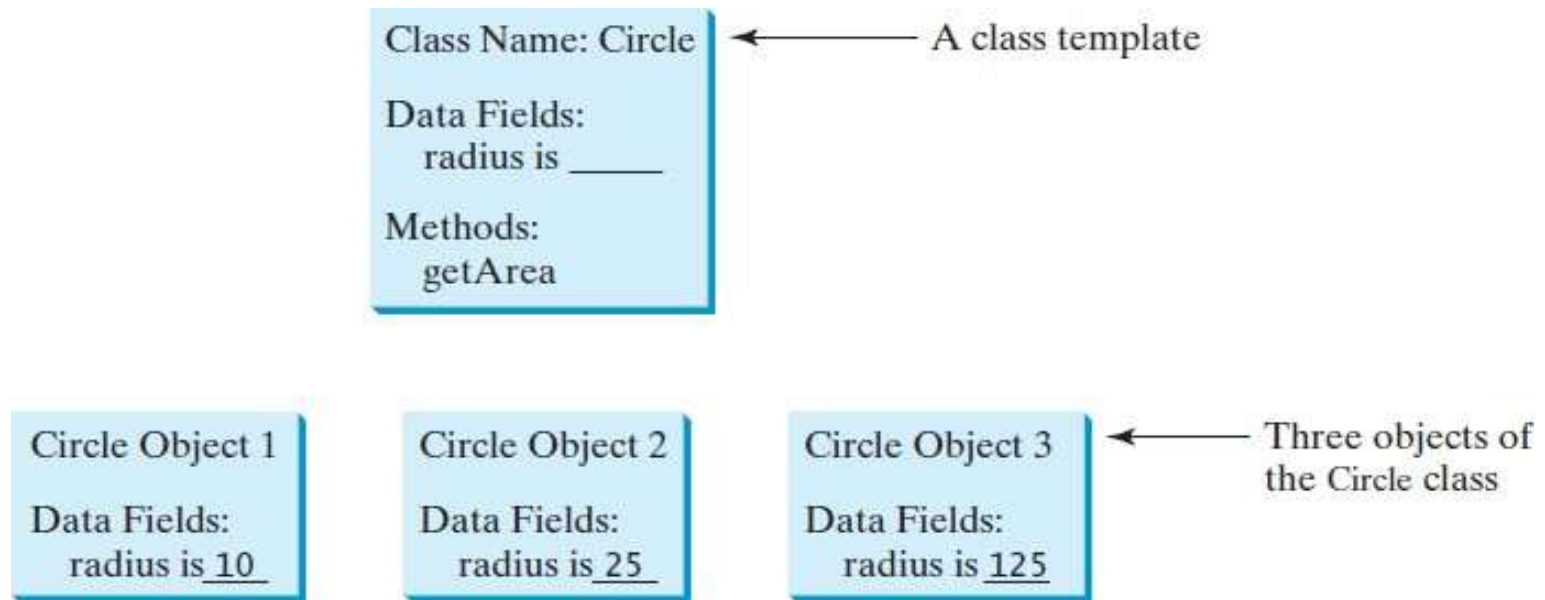


Behavior

- Behavior dalam lingkup class (dapat juga dikatakan sebagai **actions**) didefinisikan dalam bentuk **methods**.



Class is a template for creating objects



Contoh struktur objek :

Benjamin as an Object

Attributes:

```
name = "Benjamin"  
address = "1, Robinson Road"  
budget = "2000"
```

Methods:

```
purchase()    {send a purchase request to a salesperson}  
getBudget()   {return budget}
```

Bernie as an Object

Attributes:

```
name = "Bernie"  
address = "18, Sophia Road"  
budget = "1000"
```

Methods:

```
purchase()    {send a purchase request to a salesperson}  
getBudget()   {return budget}
```



Contoh desain kelasnya :

Class Customer

Attributes:

name

address

budget

Methods:

purchase() {send a purchase request to a salesperson}

getBudget() {return budget}

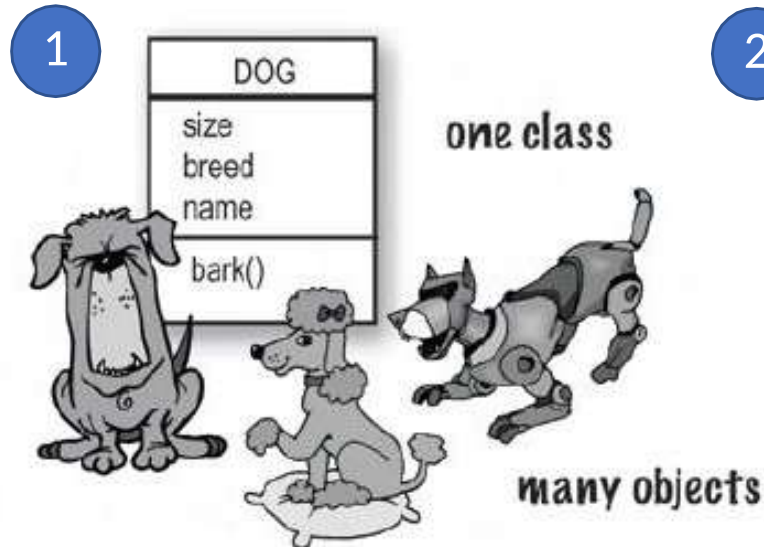
Latihan (10 menit)

- Silahkan tentukan minimal 2 (dua) kelas dari permasalahan berikut, dan tuliskan apa saja contoh state dan behaviornya.
 - E-commerce
 - Social Media
 - ATM
 - Akademik Polban
 - Etc ?



Contoh Implementasi Class :

Abstraksi kelas



Implementasi kelas

2

```
class Dog {  
    int size;  
    String breed;  
    String name;  
  
    void bark() {  
        System.out.println("Ruff! Ruff!");  
    }  
}
```

instance variables

a method

DOG
size breed name
bark()

3 Instansiasi Objek

```
class DogTestDrive {  
    public static void main (String[] args) {  
        Dog d = new Dog();  
        d.size = 40;  
        d.bark();  
    }  
}
```

make a Dog object

use the dot operator (.) to set the size of the Dog and to call its bark() method

dot operator

Class Implementation

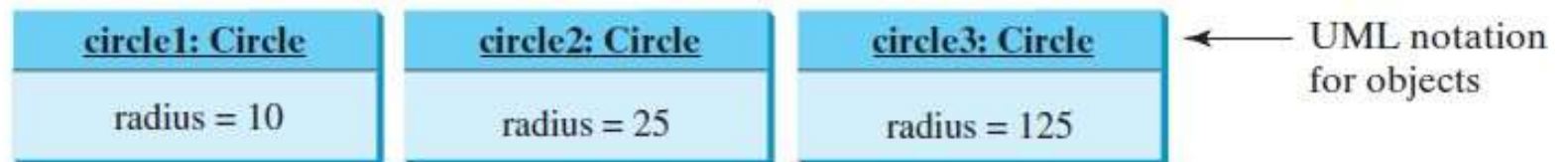
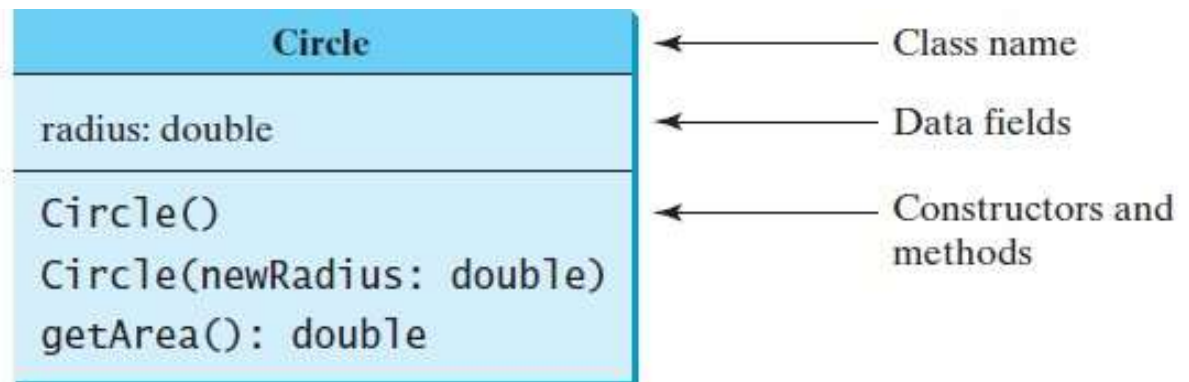
```
class Circle {  
    /** The radius of this circle */  
    double radius = 1.0;   
  
    /** Construct a circle object */  
    Circle() {  
    }  
  
    /** Construct a circle object */  
    Circle(double newRadius) {  
        radius = newRadius;  
    }  
  
    /** Return the area of this circle */  
    double getArea() {  
        return radius * radius * Math.PI;  
    }  
}
```

← Data field

← Constructors

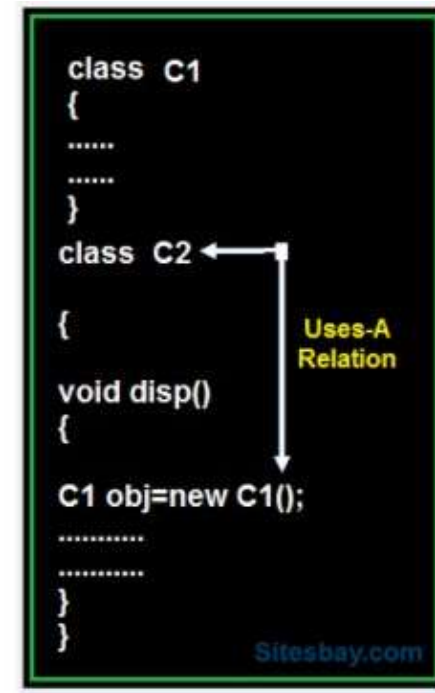
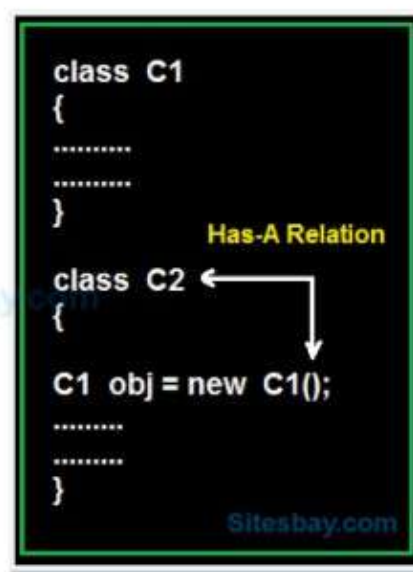
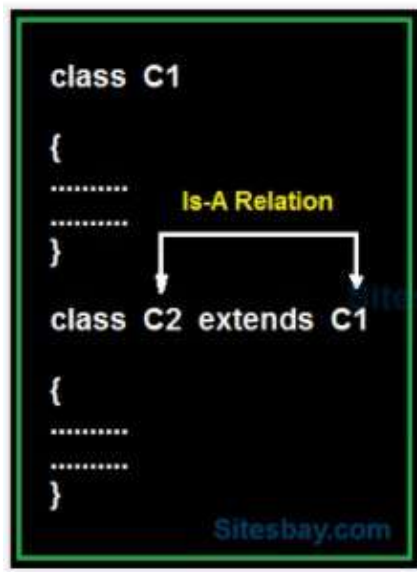
← Method

UML Class Diagram



Relationship between Class

- Dependence (“use – a”)
- Aggregation (“has – a”)
- Inheritance (“is – a”)



Contoh Relasi dependence

```
class Employee
{
    float salary=30000;
    void disp(){
        System.out.println("Salary : "+ salary);
    }
}

class Developer
{
    public static void main(String args[])
    {
        Employee e = new Employee();
        e.disp();
    }
}
```

4. Enkapsulasi (pembungkusan)

- Enkapsulasi merupakan salah satu konsep OOP yang digunakan untuk **membungkus (bundling)** sekumpulan attributes and methods kedalam satu objek dan **menyembunyikan (information hiding)** detail implementasinya dari objek lain.
- Enkapsulasi terdiri dari dua konsep, yaitu :
 1. Bundling
 2. Information Hiding



Keuntungan Enkapsulasi

- Memudahkan proses maintenance (**maintainability**)
- Perubahan kode bersifat independent
- Meningkatkan **usability**
- Meningkatkan **extensibility**



To achieve encapsulation in Java

- Declare variable of class as **private**.
- Provide **public setter** and **getter** methods to modify and view variables values.



Contoh Enkapsulasi :

```
/* File name : EncapTest.java */
public class EncapTest {
    private String name;
    private String idNum;
    private int age;

    public int getAge() {
        return age;
    }

    public String getName() {
        return name;
    }

    public String getIdNum() {
        return idNum;
    }
}
```

```
    public void setAge( int newAge) {
        age = newAge;
    }

    public void setName(String newName) {
        name = newName;
    }

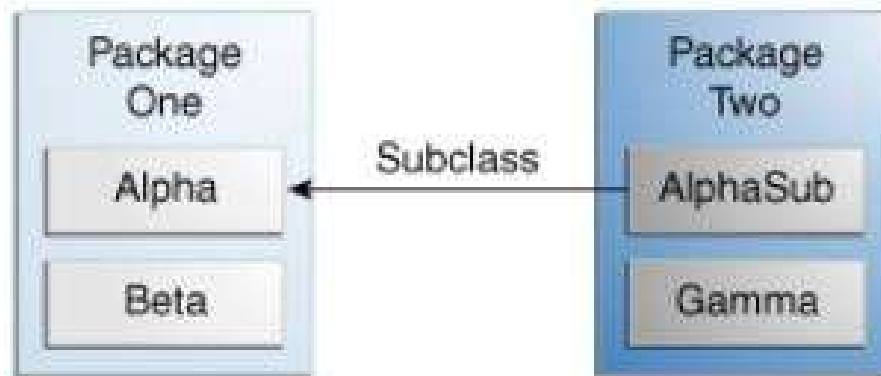
    public void setIdNum( String newId) {
        idNum = newId;
    }
}
```

Visibility (Access Modifier)

Access Levels

Modifier	Class	Package	Subclass	World
<code>public</code>	Y	Y	Y	Y
<code>protected</code>	Y	Y	Y	N
<i>no modifier</i>	Y	Y	N	N
<code>private</code>	Y	N	N	N

Contoh implementasi Access Modifier



Visibility

Modifier	Alpha	Beta	Alphasub	Gamma
public	Y	Y	Y	Y
protected	Y	Y	Y	N
<i>no modifier</i>	Y	Y	N	N
private	Y	N	N	N

Quiz

- Quiz Session



Kesimpulan :

- Object merupakan sesuatu yang nyata / tidak nyata yang memiliki **state** (attributes) dan **behaviour** (methods).
- Komunikasi antar objek dilakukan dengan pertukaran pesan (**sending message**).
- Class merupakan **blueprint** dari sebuah objek
- Message merupakan suatu pemanggilan method (**method call**) dari pengirim pesan (**sender**) kepada penerima pesan (**receiver**).
- Enkapsulasi merupakan suatu konsep OOP yang memiliki dua konsep, yaitu (1) **bundling** dan (2) **information hiding**.



Questions



References :

1. Object oriented programming and Java 2nd Edition – Chapter 1& 2
2. Head First Java 2nd Edition
3. Core Java Volume I – 10th Edition



Tugas 1

Object, Class & Encapsulation



Tugas 1:

- Buatlah contoh interaksi kelas pada java di Netbeans / Eclipse (minimal 2 kelas)
 - (catatan : tema kelas bebas, boleh mengambil dari kasus nyata / tidak nyata, contoh : game, kampus, toko, dll)
- Jenis Relationship yang digunakan adalah dependences (“use – a”)
- Gunakan enkapsulasi untuk membungkus **attributes**-nya !
- Copykan **source code** & **screenshot hasil implementasi**-nya pada word



Contd..

- Beri nama file “**tugas01_npm_nama.docx**” atau “**tugas01_npm_nama.pdf**”.
- Kumpulkan pada e-learning paling lambat 1hari sebelum perkuliahan berikutnya.